KAFKA AT THE EDGE.

Streaming, where the network is optional.



Speaker info.

Avinash Upadhyaya

01 Platform engineer @ platformatory.io

02 Confluent Community Catalyst

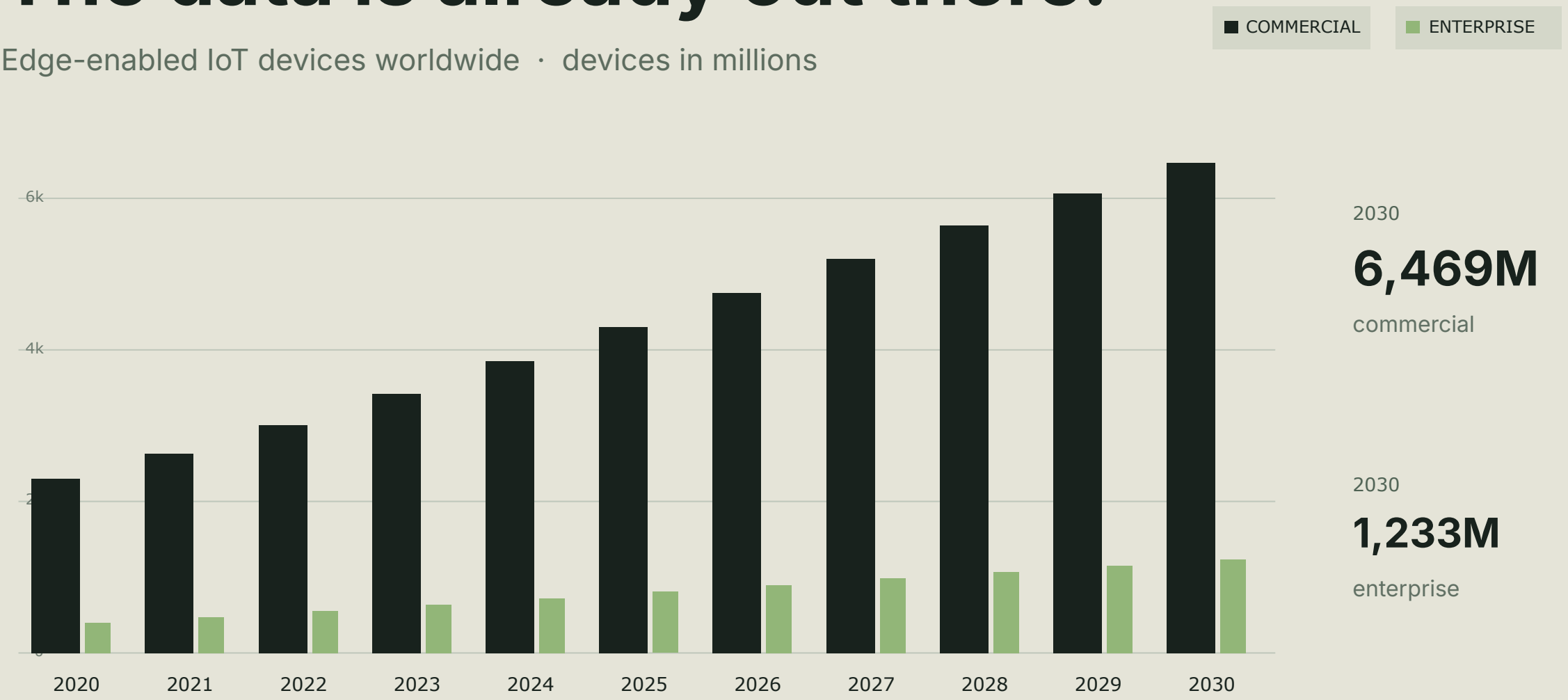
03 Occasional open source contributor to Cloud Native projects (k8s, ArgoCD, Tekton, Litmus, etc)

04 Meetup organizer for Bengaluru Streams, Kafka, Pinot, Grafana and Docker



The data is already out there.

Edge-enabled IoT devices worldwide · devices in millions



2030

6,469M

commercial

2030

1,233M

enterprise

Source: Market.us scoop


AI AGENTS
LIGHTYEAR
AI AGENTS EVERYWHERE
imgflip.com

The edge is the final frontier for streaming.

All the fallacies of distributed systems, plus:

“Incremental processing, continual learning and real-time inference will be the trifecta of emerging edge applications”

- constrained devices
- flakier networks
- very expensive bandwidth costs
- Wild west of legacy protocols
- Large volumes of data — low signal to noise ratio
- Edge sprawl
- Cost constraints

Two standards. Different jobs.

MQTT

Great at sleepy connections.

A lot of the edge world uses MQTT. It is a protocol specification, not one implementation; real-world broker capabilities vary.

Efficient telemetry, but commonly paired with proxies and Kafka when volumes, storage or downstream streaming grow.

Apache Kafka

Great at data-intensive streaming.

The de-facto platform and protocol standard for data-intensive streaming workloads.

Kafka-compatible distributed systems are usually consensus-oriented, node-heavy, and built for stable connectivity.

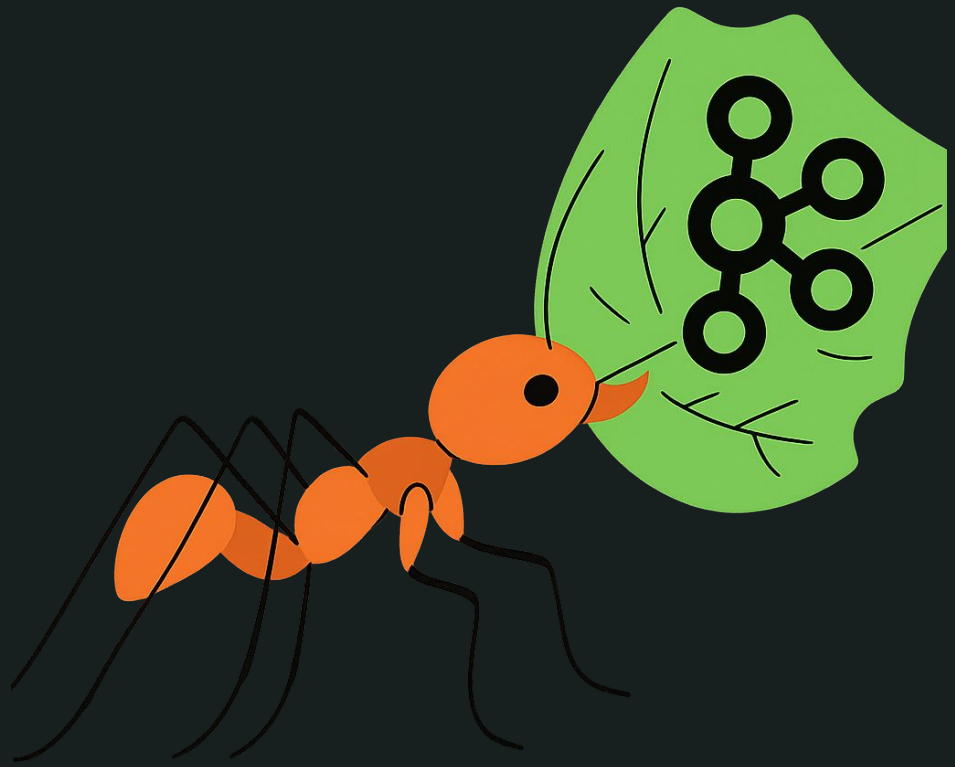
The cloud has streaming.
The edge has workarounds.



> *“There’s presently no comprehensive solution for streaming at the edge, except for naïve approaches that shoehorn a distributed system into constrained devices.”*

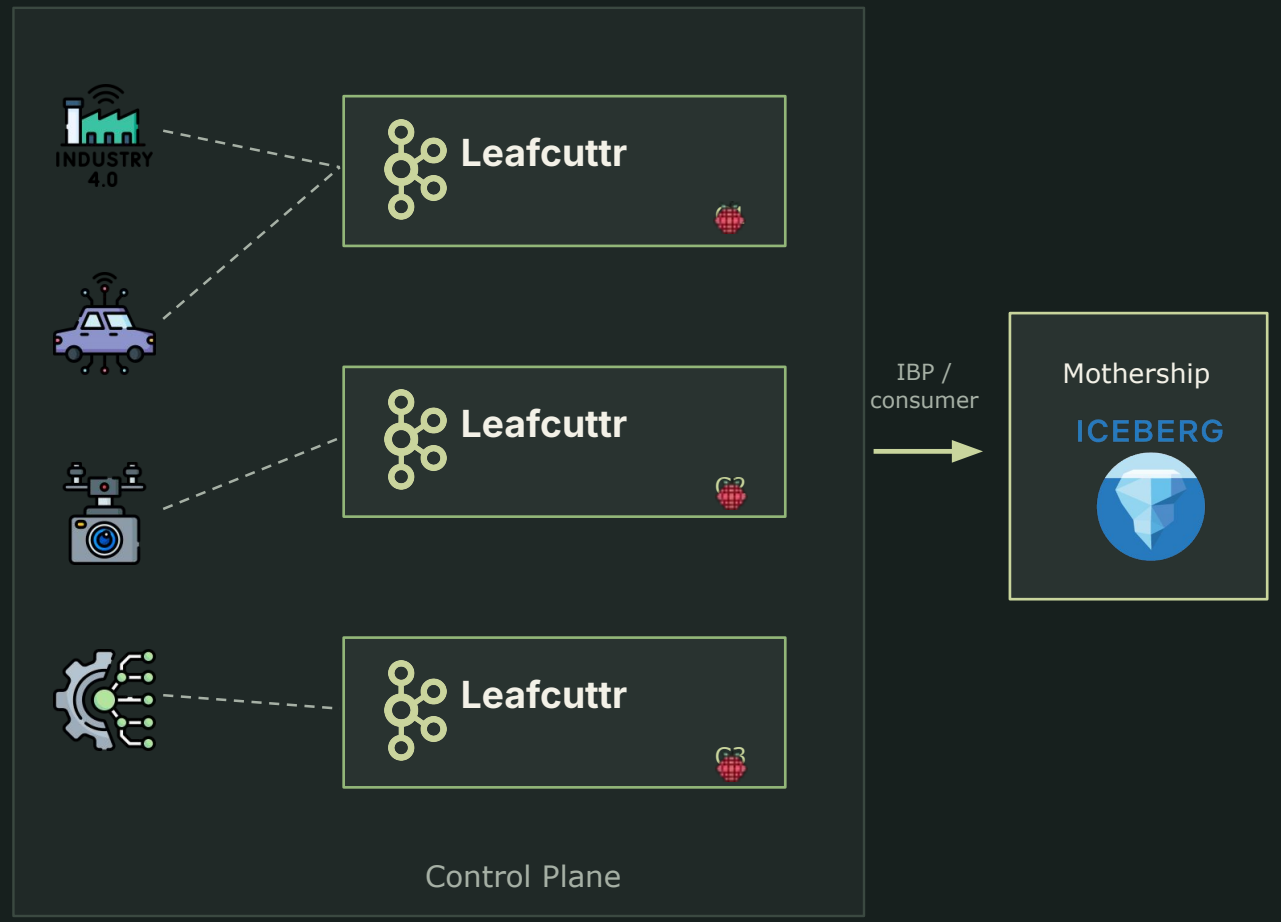
Leafcuttr = Edge native streaming

- Small, powerful Kafka-compatible stream broker
- Designed for high fan-in
- Asynchronous replication
- Flaky networks and intermittent connectivity
- Buffer → Aggregate → Forward

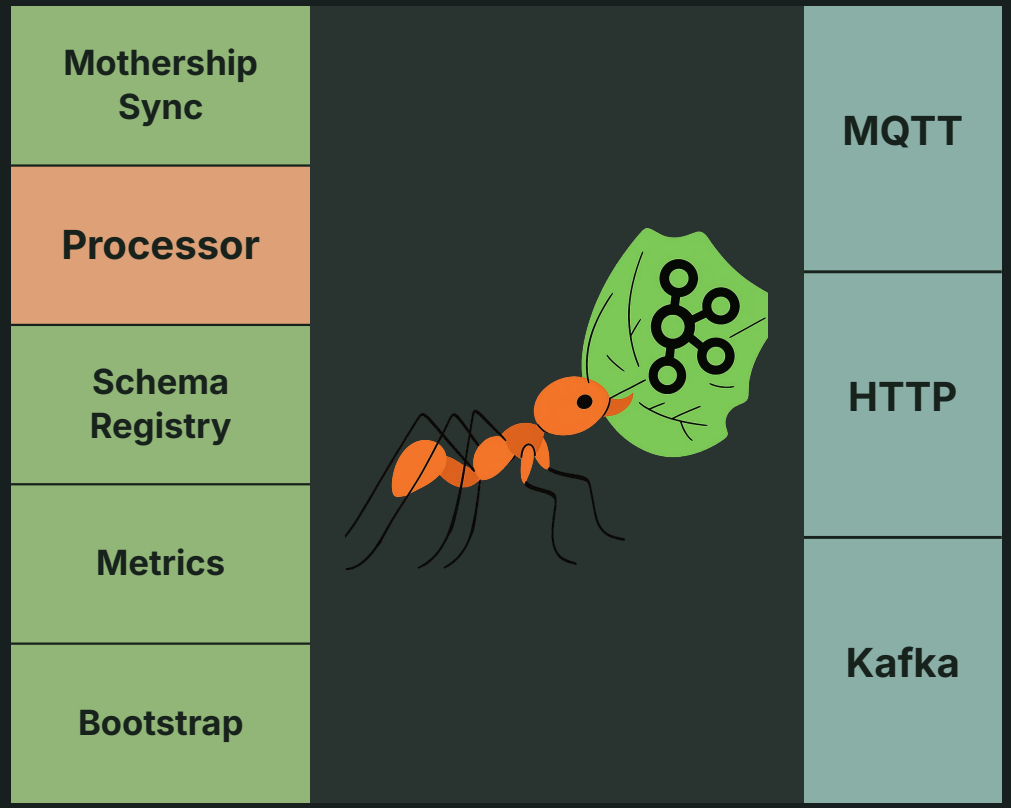


Leafcuttr = Edge native streaming

- Small and powerful Kafka-compatible stream broker for edge environments
- Designed for high fan-in
- Async replication
- Optimized for flaky networks and intermittent connectivity
- Buffer → Aggregate → Forward



Leafcuttr = Multi Protocol Edge Gateway



- One gateway to rule it all — supports MQTT, HTTP and Kafka interfaces natively without the need for new nodes or proxies
- Built in schema registry
- Observability out of the box
- Single command bootstrap to join management plane
- Coming soon: Filter, forward and aggregate streams

Two deployment models

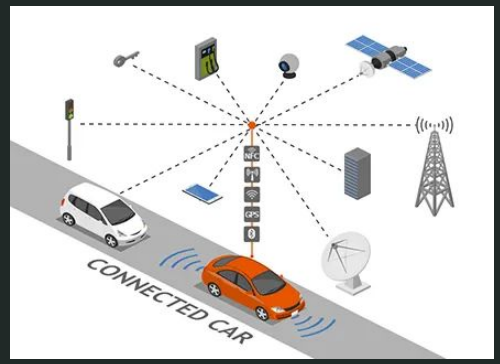
Gateway / Mesh Mode



N:1 fan-in across devices

Sidecar / Embedded Server Mode





One node inside or beside each host

Gateway / Mesh Mode

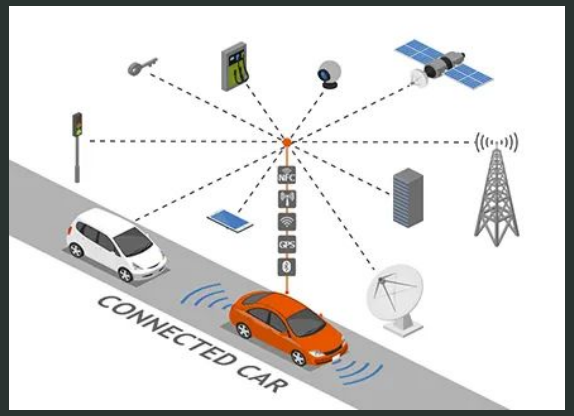


- Use-cases: Manufacturing and Industrial, Smart Buildings, Retail - POS Networks, Video & Network Surveillance
- LC Nodes deployed with N:1 fan-in.
- LC clusters configured based on visibility and connectivity.
- Each thing is sticky sessioned / pinned to an LC
- In the event of a single LC failure, thing connects to another LC (based on metadata)

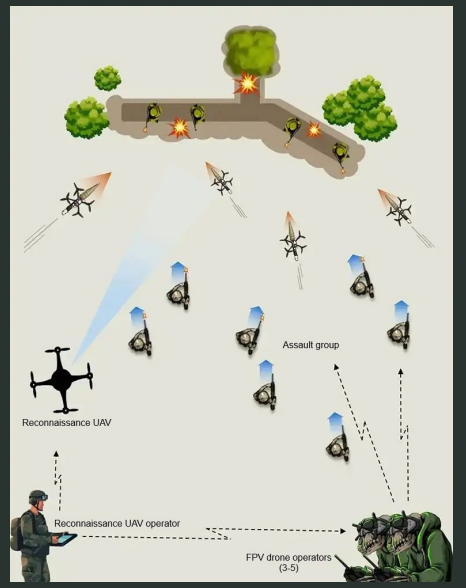
Sidecar / Embedded Server Mode

LC is an embedded server or attached sidecar — effectively single node. Shares the same fault domain as its host.









Smart Appliances · Telematics · Connected Vehicles · Robotics · Heavy Machinery

Technical Preview

Kafka-compatible. Built for the edge.

Apache Kafka fork

Leafcuttr is a fork of upstream Apache Kafka 4.x.

Kafka protocol

Fully compatible with the Kafka protocol.

Constrained hardware

Optimized for the limited CPU and memory available at the edge.

Performance and cost.

~80%

HIGHER THROUGHPUT

as characterized in the source deck

~40%

LOWER LATENCY

p50–p999 in constrained scenarios

fsync

FULL DURABILITY

as stated for benchmarked acks=1 path

Built-in aggregation, schema registry and multi-protocol ingress can remove separate proxy / SR nodes.

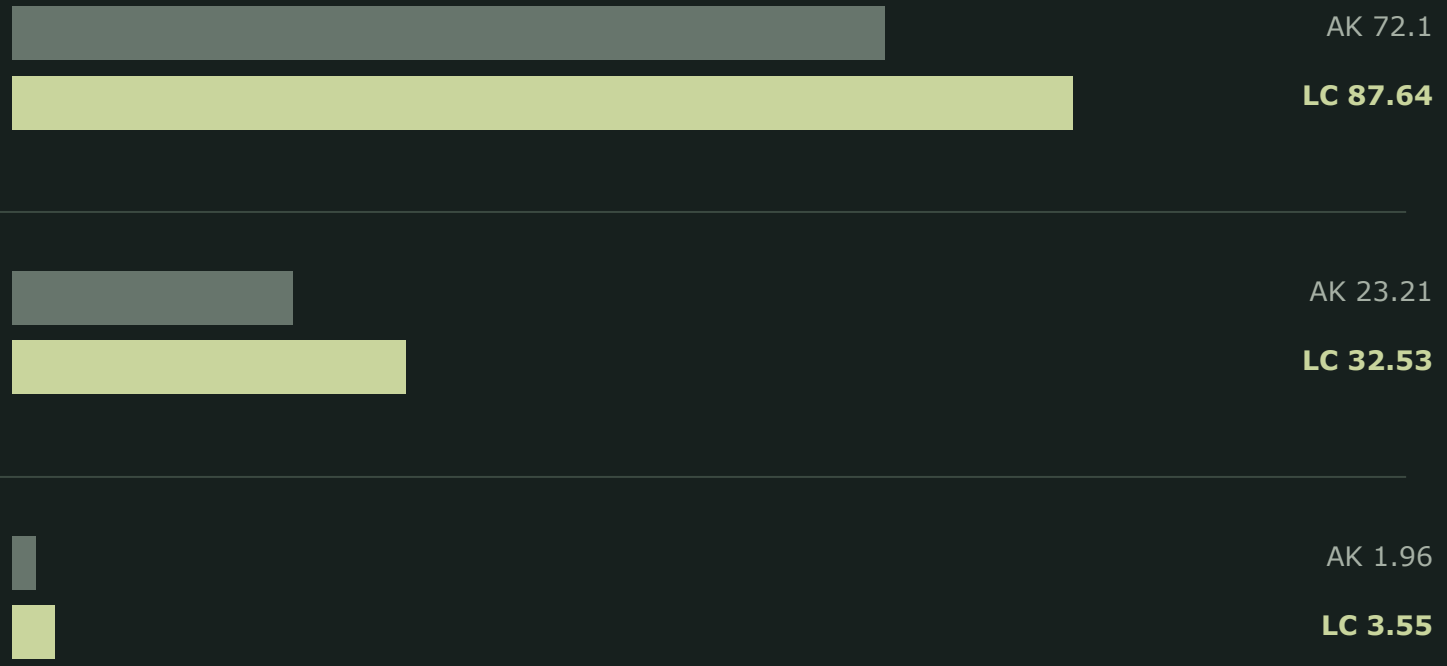
PERF.
ON SMALL HARDWARE.



512 MB RAM / 0.5 CPU

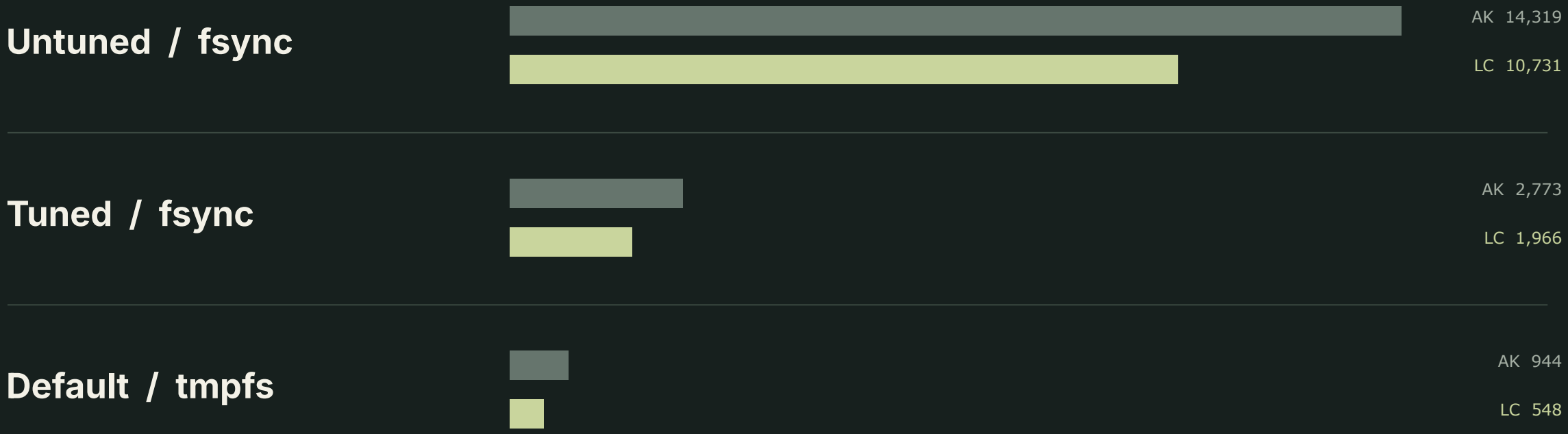
Throughput: Leafcuttr vs Apache Kafka

MB/S · HIGHER IS BETTER



Latency: Leafcuttr vs Apache Kafka

P999 LATENCY (MS) · LOWER IS BETTER



The full latency picture.

ALL VALUES IN MILLISECONDS · 512 MB RAM · 0.5 CPU

SCENARIO	MEAN	P50	P95	P99	P999
Untuned + fsync / AK	8,928.467	9,226.67	12,816	13,756	14,319
Untuned + fsync / LC	5,456.11	5,369.33	9,517.67	10,174.67	10,730.67
Tuned + fsync / AK	1,197.67	1,181	2,116.33	2,467.67	2,773
Tuned + fsync / LC	850.3	835.67	1,511.33	1,774.33	1,965.67
Default + tmpfs / AK	259.18	221	587.33	753.33	944
Default + tmpfs / LC	130.32	109.67	316.67	411.33	548

AK = APACHE KAFKA / LC = LEAFCUTTR

Native vs Docker performance.

ENGINE / MODE	THROUGHPUT MB/S	AVG MS	MAX MS	P50 MS	P99.9 MS
RedPanda Docker (acks=all; flush on)	25.11	1239	3771	1169	3721
AK 4.0.0 Docker / no flush	58.22	532.38	960	518	949
AK 4.0.0 Docker / flush interval 1	8.9	3433.56	7733	3212	7711
AK 4.0.0 Native / no flush	63.53	487	929	476	916
AK 4.0.0 Native / flush interval 1	8.44	3640	6752	3644	6741
LC Mod Native / no flush	65.59	468	944	453	933
LC Mod Native / flush interval 1	9.48	3228	5158	3139	5148
LC Mod Native / Sync Mode	12.67	2413	3769	2392	3752

SYNC MODE: +50% throughput and ~45% lower p999 vs source AK native flush case

DESIGN NOTES.

Small brokers.
Deliberate planes.



Three planes. One fleet.



How an edge node joins the fleet.

01	REGISTER	Leaf Monitor authenticates over HTTP(S) using a node token.
02	INSTALL	It bootstraps and manages native systemd services.
03	SYNC	The controller writes desired config to SQLite (JSON), then pushes updates over WebSocket.
04	REPORT	Leaf Monitor sends health and telemetry back to the controller.

MVP DESIGN NOTE Runs natively first; containerization is an option for later.

Roadmap

01	**Durable Producer API**	A swap-in Kafka producer API replacement that spills its send buffer to disk.
02	**Lightweight edge processing**	Filter, aggregate and downsample streams close to the source.
03	**Sandboxed model runtime**	Run small models in an isolated runtime at the edge.
04	**Agent Journal**	WAL & SMR for agent decision evaluations, durable execution and fault recovery.

Kafka at the edge.

Try Leafcuttr.

```
$ docker pull leafcuttr/leafcuttr:latest
```

https://leafcuttr.github.io/